Tabla de contenidos

1. Funciones que trabajan con iterables
    a. Funciones que necesitan de una función
        i. `map`
        ii. `filter`
        iii. `reduce`
        iv. Funciones *lambda* o funciones anónimas
    b. Funciones que no necesitan de una función
        i. `enumerate`
        ii. `zip`

Funciones que trabajan con iterables

Además de las funciones generadoras, también podemos encontrar funciones que no utilizan necesariamente generadores, pero aceptan el uso de generadores. En este sentido, agruparemos estas funciones en dos grupos:

1. Aquellas que necesitan de una función para ser ejecutadas.
2. Aquellas que no necesitan de una función.

Funciones que necesitan de una función

`map`

`map` recibe como parámetros una función y **al menos** un iterable. Retorna un iterador que resulta de aplicar la función sobre cada elemento del iterable por separado. Es así como `map(f, iterable)` es similar a `(f(x) for x in iterable)`.

La cantidad de iterables entregada a `map` debe corresponder con la cantidad de parámetros que recibe la función `f`. Por ejemplo, si tenemos `map(f, iterable1, iterable2)` entonces `f` debe recibir dos parámetros. Es así como `map(f, iterable1, iterable2)` es similar a `(f(x, y) for x, y in zip(iterable1, iterable2))`.

> (Inicio paréntesis
>
> La función `zip` nos permite recorrer de forma conjunta 2 o más iterables en base a la posición de sus elementos. Entraremos en más detalle sobre esta función en la sección `zip`.
>
> Fin paréntesis)

En resumen, la función `map` presenta la siguiente sintaxis `map(función, iterable1, ...)`, donde:

- `función`: denota una función de Python o, en general, cualquier invocable de Python. Esto incluye funciones integradas y definidas por el usuario, clases, métodos de instancia y clase, y más.

- `iterable`: es cualquier iterable de Python válido, como una lista, una tupla y un set.

La función `map()` aplicará el función a cada elemento en el iterable.

Imagen obtenida de Swift unboxed.

Veamos algunos ejemplos:

1. Tenemos una lista de *strings*, donde queremos colocar cada uno en minúsculas:

```
def lower(string: str) -> str:
    return string.lower()


strings = ['Señores pasajeros', 'Disculpen', 'mi', 'IntencIÓN', 'no',
'Es', 'MolEstar']
mapeo = map(lower, strings)

print(', '.join(mapeo))
```

```
señores pasajeros, disculpen, mi, intención, no, es, molestar
```

2. Tenemos dos o más listas de números y queremos, a partir de esos números, calcular otro:

```
def polinomio_1(x: int, y: int) -> int:
    return x ** 2 + y ** 2

def polinomio_2(x: int, y: int, z: int) -> int:
    return x + y ** 2 + z ** 3


a = [1, 2, 3, 4]
b = [17, 12, 11, 10]
c = (-1, -4, 5, 9)

mapeo_1 = map(polinomio_1, a, b)
mapeo_2 = map(polinomio_2, a, b, c)

print(list(mapeo_1))
print(list(mapeo_2))
```

```
[290, 148, 130, 116]
[289, 82, 249, 833]
```

Notar que la cantidad de elementos que procesa la función en un `map` corresponde a la cantidad que tiene el iterable más pequeño:

```
def suma(x: int, y: int) -> int:
    return x + y
```

```
a = range(1, 2233)
b = [100, 101, 102]

mapeo = map(suma, a, b)
list(mapeo)
```

```
[101, 103, 105]
```

`filter`

`filter(f, iterable)` recibe como parámetros una función que retorna `True` o `False` (o función *booleana*), y un iterable. Retorna un generador que entrega aquellos elementos del iterable donde la función `f` retorna `True`.

Se puede ver que `filter(f, iterable)` es equivalente a `(x for x in iterable if f(x))`.

```
from typing import Generator


def fibonacci(límite: int) -> Generator:
    a, b = 0, 1
    for _ in range(límite):
        yield a
        a, b = b, a + b

def es_impar(número: int) -> bool:
    return número % 2 != 0

def es_par(número: int) -> bool:
    return número % 2 == 0


filtrado_impares = filter(es_impar, fibonacci(10))
print(list(filtrado_impares))

filtrado_pares = filter(es_par, fibonacci(10))
print(list(filtrado_pares))
```

```
[1, 1, 3, 5, 13, 21]
[0, 2, 8, 34]
```

Otro ejemplo, en el que se entrega un *set* a `filter`:

```
def menor_que_10(número: int) -> bool:
    return número < 10
```

```
set_filtrado = filter(menor_que_10, {100, 1, 5, 9, 91, 1})
print(list(set_filtrado))
```

```
[1, 5, 9]
```

`reduce`

Vamos a explicar la idea del `reduce` con un ejemplo de cálculo manual. Imaginemos que tenemos una secuencia con números, y que queremos obtener la suma de ellos. También supongamos que nos complica sumar más de dos números a la vez.

```
lista = [1, 2, 3, 4, 5, 6]
```

Si hicieramos esta suma en forma procedural, lo que probablemente haríamos es sumar los dos primeros elementos, guardar el resultado, y ese resultado sumarlo con el siguiente elemento. Y así sucesivamente:

- $1+2=3$
- $3+3=6$
- $6+4=10$
- $10+5=15$
- $15+6=21$

El resultado final es **21**. Ahora supongamos que no necesariamente queremos sumar los números de a pares, sino que aplicar una función cualquiera `f`:

- $f(1,2)=a$
- $f(a,3)=b$
- $f(b,4)=c$
- $f(c,5)=d$
- $f(d,6)=e$

En este caso, el resultado final es **e**. Reemplazando las variables, nuestro cómputo fue:

$$f\Big(f\big(f\big(f\big(f(1,2),3\big),4\big),5\big),6\Big)$$

Esa es exactamente la idea detrás del `reduce`. Esta operación consiste en aplicar sucesivamente una función `f(x, y)`, donde `x` es el resultado acumulado e `y` es un elemento de la secuencia. Esto *reducirá* el iterable a un sólo resultado.

Entonces, `reduce(f, iterable)` recibe una función que toma dos valores y un iterable. Retorna lo que resulta de aplicar la función `f` al iterable `[s1, s2, s3, ..., sn]` de la siguiente forma: `f(f(f(f(s1, s2), s3), s4), s5), ...`.

Podemos ver que funciona muy bien para la suma que habíamos propuesto al principio:

```
from functools import reduce


def suma(x: int, y: int) -> int:
    return x + y


reduce(suma, lista)
```

```
21
```

Finalmente, es posible agregar un inicializador al `reduce`. Este inicializador será el primer elemento que la función procese.

A continuación, un ejemplo donde genera la representación como *string* de una lista de números y se utiliza el inicializador para agregar una etiqueta a esta representación:

```
from typing import Any


def concatenar_textos(x: Any, y: Any) -> str:
    return f'{x}->{y}'


lista = [1, 2, 3, 4, 5]

reduce_sin_inicializador = reduce(concatenar_textos, lista)
reduce_con_inicializador = reduce(concatenar_textos, lista,
'[Inicio]')

print(f'Sin inicializador: {reduce_sin_inicializador}')
print(f'Con inicializador: {reduce_con_inicializador}')
```

```
Sin inicializador: 1->2->3->4->5
Con inicializador: [Inicio]->1->2->3->4->5
```

Ahora, veamos algunos ejemplos aplicados:

1. Aplanamiento de listas

   Consideremos que tenemos una lista con más listas dentro, y queremos juntar todos los elementos en orden en una gran lista. Podemos hacer eso con `reduce`.

```
def concatenar_listas(x: list, y: list) -> list:
    return x + y


lista_con_listas = [[1, 2], [3, 4], [5, 6], [7, 8, 9]]
```

```
lista_aplanada = reduce(concatenar_listas, lista_con_listas)
lista_aplanada
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

1. Intersección o unión de varios *sets*

   Consideremos que tenemos varios *sets* en una lista, de los cuales queremos obtener su intersección o unión:

```
def operación_unión(x: set, y: set) -> set:
    return x | y

def operación_intersección(x: set, y: set) -> set:
    return x & y


conjuntos = [{3, 5, 1}, {4, 3, 1}, {1, 2, 5}, {9, 5, 4, 1}]

resultado_unión = reduce(operación_unión, conjuntos)
resultado_intersección = reduce(operación_intersección, conjuntos)

print("Unión:", resultado_unión)
print("Intersección:", resultado_intersección)
```

```
Unión: {1, 2, 3, 4, 5, 9}
Intersección: {1}
```

1. Duplicación de elementos

   Supongamos que queremos crear una cantidad de elementos dependiendo del valor que posean:

```
def duplicar_elementos(x: int, y: int) -> int:
    # Recuerda que el * se usa aquí para desempaquetar valores
    return *x, *(y for _ in range(y))

reduce(duplicar_elementos, lista, [])
```

```
(1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5)
```

1. Cálculo de mínimos o máximos

   Se puede obtener el máximo en una colección usando `reduce`. Hagámoslo sobre la unión de conjuntos que acabamos de obtener:

```
# Hecho así sólo como ejemplo pedagógico
def máximo(x: int, y:int) -> int:
    if x >= y:
```

```
        return x
    return y


reduce(máximo, resultado_unión)
```

```
9
```

No obstante, es más limpio usar simplemente la función `max` o `min` que nos provee Python:

```
max(resultado_unión)
```

```
9
```

Precauciones ⚠

1. Cantidad de elementos de la secuencia a reducir

   Cuando la secuencia que se entrega a `reduce` tiene sólo un elemento, la operación retornará sólo ese elemento sin aplicar la función.

```
def suma(x: int, y: int) -> int:
    return x + y


reduce(suma, [1])
```

```
1
```

Como se comentaba anteriormente:

> Es posible agregar un inicializador al `reduce`. Este inicializador será el primer elemento que la función procese.

Normalmente se define como el objeto que no altera el resultado de nuestra función, que en el caso de la suma es el número 0.

```
reduce(suma, [1], 0)
```

```
1
```

Si la secuencia entregada es vacía y no entregamos un valor de inicialización, se lanza una excepción:

```
try:
    reduce(suma, [])
except TypeError as error:
    print('TypeError:', error)
```

```
TypeError: reduce() of empty iterable with no initial value
```

Mientras que añadiendo un inicializador, el proceso termina exitosamente:

```
reduce(suma, [], 0)
```

```
0
```

1. Operaciones no conmutativas

   Es necesario que tomes en cuenta que en ciertas operaciones el resultado va a depender del orden en que se encuentren los elementos en la colección. Esto es así por ejemplo en la división, donde $\frac{x}{y} \neq \frac{y}{x}$, mientras que en operaciones como la suma el orden no es relevante pues $x+y=y+x$.

   Veamos con un ejemplo qué pasa cuando el orden de los elementos cambia en una operación sensible al orden:

```
def división(x: int, y: int) -> float:
    return x / y


números = [3, 5, 7, 9, 11]
reduce(división, números)
```

```
0.0008658008658008659
```

Acá invertiremos el orden de los números:

```
reduce(división, números[::-1])
```

```
0.01164021164021164
```

1. Reducir *sets* u otros iterables no ordenados

   En este caso hay que tener cuidado cuando la operación que se haga dependa del orden de los elementos, pues el resultado podría no ser el esperado.

   En el ejemplo, se tiene un iterador con varias palabras que queremos concatenar con un `reduce`. Vemos que el orden final dista del orden que se declaró al inicio, pues esta estructura no es ordenada.

```
from random import shuffle
from typing import Self


class IteradorListaPalabrasDesordenadas:
    def __init__(self, iterable: list) -> None:
        self.iterable = iterable.copy()
```

```
    def __iter__(self) -> Self:
        shuffle(self.iterable)
        return self

    def __next__(self) -> str:
        if not self.iterable:
            raise StopIteration("Llegamos al final")
        else:
            valor = self.iterable.pop(0)
            return valor


def concatenar_textos(x: str, y: str) -> str:
    return f"{x} {y}"


# Ejecutemos este código múltiples veces para apreciar cómo cambia el
orden en cada ejecución
for i in range(1, 6):
    palabras = IteradorListaPalabrasDesordenadas(['casa', 'mar',
'ventana', 'roca', 'piso'])
    texto = reduce(concatenar_textos, palabras)
    print(f'{i}. {texto}')
```

```
1. casa mar piso roca ventana
2. piso casa roca mar ventana
3. piso ventana roca casa mar
4. piso roca ventana mar casa
5. casa mar roca ventana piso
```

Funciones *lambda* o funciones anónimas

Hasta el momento, para cada función `map`, `filter` o `reduce` que hemos implementado, hemos tenido que escribir una función que pueda ser utilizada en dicho caso, pese a la complejidad de las funciones en sí.

En este contexto, las funciones *lambda* o funciones anónimas nos serán de especial utilidad, ya que nos permitirá hacer uso de `map`, `filter` y `reduce` sin la necesidad de tener que definir funciones usando el `def`.

Pero antes de explicar qué son las funciones *lambda* y cómo funcionan, necesitamos hablar sobre **cómo se tratan las funciones en Python**. En el caso de Python, se dice que el lenguaje tiene **funciones de primera clase** (*first-class functions*), es decir, que las funciones son tratadas como cualquier otra variable (objeto). Esto no es así en otros lenguajes como Java.

El hecho que las funciones sean de primera clase tiene algunas consecuencias, como:

> **1. Las funciones pueden ser asignadas a una variable, y luego usar esa variable igual que la función.**

```python
def suma(x: int, y: int) -> int:
    return x + y


adición = suma

# Ambas son la misma función
print(adición)
print(suma)

# Y por lo tanto entregan el mismo resultado
print(suma(3, 5))
print(adición(3, 5))
```

```
<function suma at 0x000001A6F136A020>
<function suma at 0x000001A6F136A020>
8
8
```

2. Las funciones pueden ser pasadas como parámetro a otras funciones.

```python
from typing import Callable


def saludar_señora(nombre: str) -> str:
    return "Señora " + nombre

def saludar_joven(nombre: str) -> str:
    return "Joven " + nombre

def saludar_tarde(función_saludo: Callable, nombre: str) -> str:
    return "Buenas tardes " + función_saludo(nombre)


print(saludar_tarde(saludar_señora, "Valeria"))
print(saludar_tarde(saludar_joven, "Bon"))
```

```
Buenas tardes Señora Valeria
Buenas tardes Joven Bon
```

En este contexto, **las funciones *lambda*** son una forma alternativa de definir funciones en Python. Además de su nombre griego, no hay nada intimidante en ellas. Veamos un ejemplo de cómo definirlas:

```python
# Definimos una función lambda a retornar
def retornar_sucesor():
    return lambda x: x + 1

# guardamos una función lambda para luego imprimir su nombre.
sucesor = retornar_sucesor()
```

```
# Lo anterior es equivalente a escribir esta función:
def sumar_uno():
    def sucesor(x):
        return x + 1
    return sucesor

sucesor_funcion = sumar_uno()

print("2 + 1 =", sucesor(2))
print("2 + 1 =", sucesor_funcion(2))
```

```
2 + 1 = 3
2 + 1 = 3
```

```
# Aquí tenemos otro ejemplo
def restar_por(n: int):
    return lambda x: x - n

restar_3_a = restar_por(3)

# Lo anterior es equivalente a escribir esta función:
def disminuir_por(n: int):
    def disminuir(x):
        return x - n
    return disminuir

disminuir_3_a = disminuir_por(3)


print("5 - 3 = ", restar_3_a(5))
print("5 - 3 = ", disminuir_3_a(5))
```

```
5 - 3 =  2
5 - 3 =  2
```

Como se puede observar, la sintaxis consiste en `lambda <parámetros>: <valor a retornar>`. En estas funciones no se necesita la sentencia `return` en su interior, puesto que la operación que se coloca a la derecha de los dos puntos (`:`) es el valor que se devolverá.

Una característica que distingue a las funciones *lambda* es que **pueden ser definidas en forma anónima**, es decir, funciones que no reciben un nombre específico.

```
sucesor.__name__
```

```
'<lambda>'
```

```
restar_3_a.__name__
```

```
'<lambda>'
```

```
disminuir_3_a.__name__
```

```
'disminuir'
```

Estas funciones *lambda* pueden ser vistas como una forma alternativa para definir funciones que no requiere declararlas como variable (no es necesario usar el `def`), requiere de menos código para escribirlas, y son anónimas al no poseer un nombre asignado.

Estas características que presentan las *lambda functions*, combinan bien con las funciones que vimos anteriormente: `map`, `filter`, `reduce`.

Veamos cómo se verían algunos de los ejemplos anteriores, pero utilizando *lambda functions*:

```
strings = ['Señores pasajeros', 'Disculpen', 'mi', 'IntencIÓN', 'no',
'Es', 'MolEstar']
mapeo = map(lambda x: x.lower(), strings)

print(', '.join(mapeo))
```

```
señores pasajeros, disculpen, mi, intención, no, es, molestar
```

```
set_filtrado = filter(lambda x: x < 10, {100, 1, 5, 9, 91, 1})
print(list(set_filtrado))
```

```
[1, 5, 9]
```

```
lista = [1, 2, 3, 4, 5]

reduce_sin_inicializador = reduce(lambda x, y: f'{x}->{y}', lista)
reduce_con_inicializador = reduce(lambda x, y: f'{x}->{y}', lista,
'[Inicio]')

print(f'Sin inicializador: {reduce_sin_inicializador}')
print(f'Con inicializador: {reduce_con_inicializador}')
```

```
Sin inicializador: 1->2->3->4->5
Con inicializador: [Inicio]->1->2->3->4->5
```

Funciones que no necesitan de una función

`enumerate`

`enumerate()` entrega una especie de generador que retorna tuplas, donde el primer objeto en cada tupla es el indice y el segundo es el ítem original. Por ejemplo, si queremos iterar sobre una lista, y necesitamos obtener tanto el índice como su valor, una forma poco *pythonic* de hacer esto sería la siguiente:

```
lista = ["a","b","c","d"]

for indice in range(len(lista)):
```

```
    elemento = lista[indice]
    print(f"{indice}: {elemento}")
```

```
0: a
1: b
2: c
3: d
```

La función `enumerate` nos permite hacer exactamente mismo, pero de una forma más elegante y *pythonic*:

```
for indice, elemento in enumerate(lista):
    print(f"{indice}: {elemento}")
```

```
0: a
1: b
2: c
3: d
```

Notar que la función `enumerate` retorna un objeto de tipo `enumerate`, que se comporta de manera similar a un generador, por lo que puedes usar `next` para acceder a sus elementos, y eso es lo que permite utilizarlo en un `for`.

`zip`

Toma dos o más secuencias o iterables y retorna un iterador que entrega tuplas, donde cada tupla está formada por los elementos i-ésimos de cada una de las secuencias o iterables. La cantidad de elementos que retorna este iterador es igual al menor de los largos de las secuencias o iterables.

De manera similar a `enumerate`, `zip` retorna un objeto de tipo `zip` que se comporta de manera similar a un generador, por lo que también se puede usar `next` para acceder a sus elementos. Además de tener los mismos beneficios que un generador regular (ocupa poco espacio y "genera" los elementos a medida que son requeridos).

A modo de ejemplo, consideremos que tenemos una tupla con los *headers* (o nombres de columnas) de un `CSV` y una tupla con los datos de una persona en particular. Queremos obtener una lista con tuplas, donde en cada una aparezca el *header* con su valor:

```
columnas = ("nombre", "apellido", "email")
persona = ("Juan", "Perez", "jp1@hotmail.com")

list(zip(columnas, persona))
```

```
[('nombre', 'Juan'), ('apellido', 'Perez'), ('email',
'jp1@hotmail.com')]
```

Si ampliamos nuestro ejemplo con una lista de tuplas de personas:

```
columnas = ("nombre", "apellido", "email")
personas = [
            ("Juan", "Perez", "jp1@hotmail.com"),
            ("Gonzalo", "Aldunate", "gan@gmail.com"),
            ("Alberto", "Gomez", "agomez@yahoo.com")
           ]

# El asterisco simple es para pasar la lista de personas como
argumentos separados:
# Si personas = [p1, p2, p3], entonces lo siguiente es equivalente a
zip(columnas, p1, p2, p3)
list(zip(columnas, *personas))
```

```
[('nombre', 'Juan', 'Gonzalo', 'Alberto'),
 ('apellido', 'Perez', 'Aldunate', 'Gomez'),
 ('email', 'jp1@hotmail.com', 'gan@gmail.com', 'agomez@yahoo.com')]
```

También recordemos que `zip` sólo tomará la cantidad de elementos del iterable más corto. Si quieres que tome en cuenta la lista más larga puedes ver la función `zip_longest` del paquete `itertools`.

```
columnas = ("nombre", "apellido", "email")
persona = ("Juan", "Perez", "jp1@hotmail.com", "+56123123??")

list(zip(columnas, persona))
```

```
[('nombre', 'Juan'), ('apellido', 'Perez'), ('email',
'jp1@hotmail.com')]
```

`zip` como inversa de sí misma

`zip` en conjunto con el operador `*` (usado para desempacar listas o tuplas a argumentos de una función) puede ser usado como inversa de la operación `zip`.

```
a = [1, 2, 3, 4]
b = ["a", "b", "c", "d"]

zipped = zip(a, b)
zipped = list(zipped)
print(zipped)
```

```
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd')]
```

```
unzipped = zip(*zipped)
unzipped = list(unzipped)
print(unzipped)
```

```
[(1, 2, 3, 4), ('a', 'b', 'c', 'd')]
```

En el ejemplo, al ejecutar `zip(*zipped)` el desempaquetamiento entrega los 4 iterables a `zip` **por separado**. Cada uno de estos iterables tiene largo 2. Luego, `zip` concatena todos los primeros elementos `(1, 2, 3, 4)` y luego todos los segundos elementos `('a', 'b', 'c', 'd')`, volviendo a como estaba en un principio.